

Portfolio Project Blueprint

Three Production-Grade Systems for a Java + React Developer

1. Autonomous Enterprise CRM | Agentic Workflow Platform
2. AI Kitchen Prep-Time Estimator | Rider Dispatch Sync Engine
3. Merchant Credit Scoring | Alternative Data Underwriting Platform

Prepared for a developer with a background in React, Java (Spring Boot), RDBMS and NoSQL

Introduction

This document lays out three portfolio-worthy projects, each explained first in plain language and then as a full technical build plan. All three follow the same underlying pattern used at companies like Swiggy, Zomato and Razorpay: **Java Spring Boot stays the core of the system** (APIs, security, database, business logic), while a small **Python FastAPI microservice** is added purely to run the machine-learning / AI part. You do not need to abandon Java — you are simply adding one new, well-scoped skill (Python + a bit of ML) on top of what you already know.

Each project section below includes: a simple explanation, an architecture diagram, what you need to learn, the full tech stack, and a phase-by-phase build roadmap. A combined 4-week bridge plan and a project comparison table are included at the end to help you choose where to start.

PROJECT 1

Autonomous Enterprise CRM Agentic Workflow Platform

In Simple Words

Think of this as a smart, self-driving customer support system. Normally, when a customer sends a message, a human agent reads it, checks past history, decides what to do, and replies. In this project, an AI "agent" does that thinking step automatically — it reads the message, looks up the customer's past conversations, decides the best action (like "reply directly", "escalate to a human", or "create a support ticket"), and the whole decision process is shown live on screen as a flowing diagram, almost like a subway map of the AI's thought process.

How the Pieces Talk to Each Other

React (screen) ↔ Java Spring Boot (traffic controller & database) ↔ Kafka (message queue) ↔ Python FastAPI + LangGraph (the AI brain) ↔ PostgreSQL with pgvector (memory store for past conversations).

What You Need to Learn

- **Python & FastAPI** — just enough to wrap an AI model in a simple API.
- **LangGraph / CrewAI** — frameworks that let an AI agent make multi-step decisions and remember state.
- **Vector embeddings & RAG** — converting text into numbers so the system can "search by meaning", stored via the pgvector extension in PostgreSQL.
- **Spring Kafka** — event-driven messaging so Java and Python can communicate asynchronously.
- **Spring WebSockets (STOMP)** — pushing live updates from the backend straight to the React screen.

Tech Stack

Layer	Technologies
Frontend	React, TypeScript, React Flow (for the live agent diagram), Tailwind CSS
Core Backend	Java Spring Boot, Spring Security (OAuth2/JWT), Spring Data JPA, Spring Kafka, Spring WebSocket
AI Microservice	Python, FastAPI, LangGraph, OpenAI API or local Llama-3 via Ollama
Database & Cache	PostgreSQL with pgvector, Redis (agent state caching)

Build Roadmap

Phase	What You Build
Phase 1 — Java & DB Setup	Core Spring Boot REST API to manage CRM leads, contacts and tickets, backed by PostgreSQL.
Phase 2 — Vector Database	Enable pgvector; convert customer interaction history into embeddings and store them.
Phase 3 — Python AI Microservice	FastAPI + LangGraph service: takes a customer message, searches pgvector for context, decides an action, returns JSON.
Phase 4 — Integration & Kafka	Route messages through Kafka to Java, trigger the Python service, update PostgreSQL, and push live updates to React via WebSockets.

AI Kitchen Prep-Time Estimator

Rider Dispatch Sync Engine

In Simple Words

This is the system behind apps like Swiggy or Zomato that decides exactly when to send a delivery rider to a restaurant. If the rider arrives too early, they wait around; too late, and the food gets cold. This project predicts how long the kitchen will take to prepare an order, then times the rider's dispatch so both the food and the rider are ready at almost the same moment.

How the Pieces Talk to Each Other

React + Mapbox (kitchen & rider live views) ↔ Java Spring Boot (order state machine & dispatch logic) ↔ Python FastAPI (a small ML model that predicts prep time) ↔ PostgreSQL + PostGIS (location-aware database) + Redis (live rider GPS cache).

What You Need to Learn

- **Python data basics** — Pandas & NumPy for handling tabular order data.
- **Tabular machine learning** — XGBoost or Scikit-learn to predict prep time as a number (regression).
- **PostGIS** — spatial SQL queries such as distance calculations and "find riders within X km".
- **Spring State Machine** — cleanly modelling the order lifecycle: Placed → Preparing → Rider Dispatched → Delivered.
- **Geospatial UI** — Mapbox GL JS or Leaflet in React to show riders and delivery zones on a live map.

Tech Stack

Layer	Technologies
Frontend	React, Mapbox GL JS, Recharts, Tailwind CSS
Core Backend	Java Spring Boot, Spring StateMachine, Spring Data JPA, Spring WebSockets
ML Microservice	Python, FastAPI, Scikit-learn, XGBoost, Joblib (model serialization)
Database & Cache	PostgreSQL + PostGIS, Redis (live rider coordinates)

Build Roadmap

Phase	What You Build
Phase 1 — Data & Model Training	Generate synthetic order data (item count, kitchen load, weather, time of day). Train an XGBoost regression model and save it as .pkl / .onnx.
Phase 2 — Python Inference API	FastAPI endpoint: order details in → predicted prep time in minutes out.
Phase 3 — Java Dispatch Engine	Spring Boot + PostGIS: on a new order, call the Python API for prep time, compute rider travel time, and schedule dispatch at the right moment.
Phase 4 — Frontend Sync	Two React views: a Kitchen Display System and a Live Dispatch Map with Mapbox.

Merchant Credit Scoring & Alternative Data Underwriting Platform

In Simple Words

This is a mini-fintech system that decides whether a small shopkeeper (a "merchant") deserves a business loan — without needing traditional credit history. Instead, it looks at real transaction data (like daily UPI/QR sales) to judge financial health, using an AI model to produce a credit score plus a plain-English explanation of why that score was given, similar to how Razorpay or Paytm's lending arms work.

How the Pieces Talk to Each Other

React Analytics Dashboard ↔ Java Spring Boot (ledger, security, loan workflow) ↔ Python FastAPI (XGBoost model + SHAP explainability) ↔ PostgreSQL (strict ACID database for the loan ledger) + Redis.

What You Need to Learn

- **Feature engineering** — turning raw transaction logs into meaningful numbers (e.g. 30-day average revenue, weekend spike ratio, volatility).
- **XGBoost classifier** — a binary model that predicts "likely to default" vs "likely to repay".
- **SHAP (explainability)** — a Python library that explains why the model gave a particular score, e.g. "revenue stability added +30 points".
- **Financial ledger design** — proper double-entry-style SQL tables for loan disbursal and auto-deduction repayments.

Tech Stack

Layer	Technologies
Frontend	React, TypeScript, Chart.js / Tremor, Tailwind CSS
Core Backend	Java Spring Boot, Spring Security (JWT), Hibernate/JPA, PostgreSQL
ML & Explainability	Python, FastAPI, Pandas, Scikit-learn, XGBoost, SHAP
Database & Cache	PostgreSQL (ACID loan ledger), Redis

Build Roadmap

Phase	What You Build
Phase 1 — Ledger in Java	Design the schema for merchant profiles, QR transaction history, loan applications and repayments.
Phase 2 — Feature Engineering & Training	Compute risk features from synthetic transaction logs in Python, train an XGBoost classifier, and add SHAP for reason codes.
Phase 3 — Python Risk API	FastAPI endpoint POST /score-merchant returning a credit score (300-850), approved loan limit, and SHAP reason codes.
Phase 4 — Java Integration & Frontend	Spring Boot calls the Risk API during loan applications; persist approvals in PostgreSQL; build the React portal showing offers, risk factors and repayment schedules.

Fast-Track Bridge Plan

A 4-week plan to pick up just enough Python and ML to build all three projects, without going deep into machine-learning theory.

Week	Focus Area	Goal
Week 1	Python & FastAPI basics	Learn core syntax, data structures, and build simple API endpoints (roughly 2-3 days).
Week 2	Pandas & Scikit-learn / XGBoost	Load tabular data with <code>pd.DataFrame</code> , train a model with <code>.fit()</code> , predict with <code>.predict()</code> , save with <code>joblib</code> .
Week 3	Spring Boot ↔ Microservice calls	Practice calling your Python FastAPI endpoints from Spring Boot using <code>WebClient</code> or <code>RestClient</code> .
Week 4	Database add-ons	Set up PostgreSQL with <code>pgvector</code> and <code>PostGIS</code> in Docker; practice basic vector and spatial queries.

Choosing Where to Start

Project	Best For	New Concepts Needed
1. CRM Agentic Platform	Those interested in AI agents, chatbots, and LLM-driven products	LangGraph, RAG, <code>pgvector</code> , Kafka
2. Prep-Time & Dispatch Engine	Those interested in logistics, real-time systems and maps	Regression ML, <code>PostGIS</code> , Spring State Machine
3. Merchant Credit Scoring	Those interested in fintech, risk and explainable AI	Classification ML, SHAP, financial ledger design

Recommended Order

If picking one starting point: build **Project 2 (Prep-Time Estimator)** first — it uses the simplest ML (plain regression) and is the most self-contained. Then move to **Project 3**, which adds classification and explainability (SHAP), a strong fintech-recruiter signal. Finish with **Project 1**, the most advanced, since agentic AI with LangGraph and vector search has the steepest learning curve but is currently the most in-demand skill set.